SAÉ 3.02 - Développement d'Applications Communicantes (Client/Serveur Android)

1. Introduction et Objectifs

Dans le cadre de la SAÉ 3.02, l'objectif était de concevoir une application Android complète implémentant une architecture Client/Serveur. Le projet devait répondre à plusieurs contraintes techniques fortes : la gestion des protocoles TCP et UDP, le multithreading pour gérer plusieurs clients simultanément, l'envoi de données structurées (sérialisation) et la mise en place d'une gestion stricte des droits d'accès via une authentification sécurisée.

2. Architecture Technique

Pour répondre au cahier des charges et simuler une véritable application métier, j'ai structuré mon projet autour de quatre composants majeurs :

1. **L'Authentification (LoginActivity) :** Point d'entrée de l'application. Elle gère la vérification des identifiants (simulant une base de données) et attribue les rôles (Admin ou Étudiant).
2. **Le Menu Principal (MainActivity) :** Hub central qui adapte l'interface selon le rôle de l'utilisateur connecté (affichage conditionnel des boutons).
3. **Le Serveur (ServerActivity) :** Machine écoutant sur le port 8080, capable de traiter les requêtes entrantes sans bloquer l'interface principale (Multithreading).
4. **Le Client (ClientActivity) :** Interface simplifiée permettant d'envoyer des messages via TCP ou UDP.

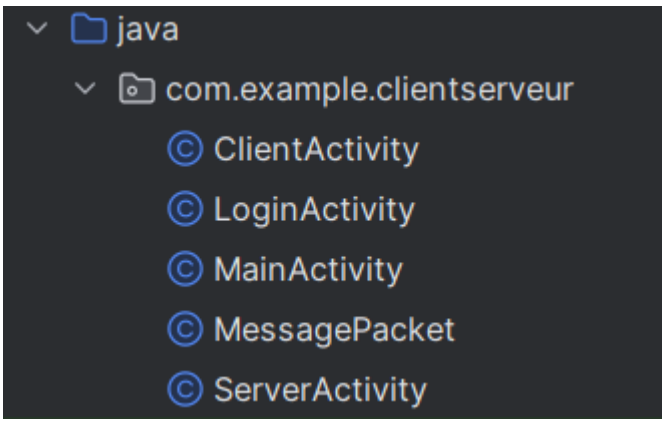

Figure 1 : Organisation des classes du projet sous Android Studio.

SAÉ 3.02 - Développement d'Applications Communicantes (Client/Serveur Android)

2.1 La structure de données (Sérialisation)

Au lieu d'envoyer de simples chaînes de caractères, j'ai créé une classe Java dédiée : MessagePacket. Elle implémente l'interface Serializable pour être transformée en flux d'octets.

Elle encapsule les informations de la transaction :

- type : Le type de requête (ex: "MESSAGE").
- user : L'identifiant de l'utilisateur (récupéré automatiquement depuis la session).
- content : Le contenu du message.

```
public class MessagePacket implements Serializable {
    no usages
    private static final long serialVersionUID = 1L;

    1 usage
    public String type;    // Ex: "MESSAGE"
    5 usages
    public String user;    // Qui envoie (Etu1, Admin...)
    4 usages
    public String content; // Le texte du message

    1 usage
    public MessagePacket(String type, String user, String content) {
        this.type = type;
        this.user = user;
        this.content = content;
    }
}
```

Figure 2 : La classe MessagePacket implémentant l'interface Serializable pour le transport réseau.

SAÉ 3.02 - Développement d'Applications Communicantes (Client/Serveur Android)

3. Démarche de Développement

3.1 Gestion des Droits et Authentification (RBAC)

J'ai implémenté un système de **Contrôle d'Accès Basé sur les Rôles (RBAC)** dès le lancement de l'application :

- **Système de Login :** L'activité LoginActivity vérifie les identifiants saisis par rapport à une liste définie :
  - **Admin :** (User: admin / Mdp: root)
  - **Étudiants :** (User: Etu1 ou Etu2 / Mdp: Roa42300)
- **Adaptation de l'Interface :** Dans le MainActivity, une vérification du booléen isAdmin est effectuée.
  - Si l'utilisateur est **Admin**, le bouton "Lancer Serveur" est visible.
  - Si l'utilisateur est **Étudiant**, ce bouton est masqué (View.GONE). Cela empêche physiquement un utilisateur non autorisé d'accéder au panneau d'administration serveur.

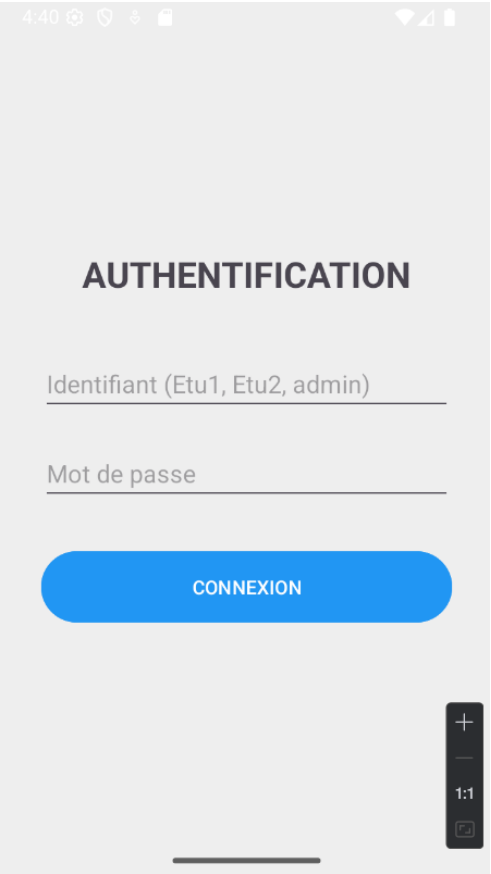

Figure 3 : Interface d'authentification sécurisée.

SAÉ 3.02 - Développement d'Applications Communicantes (Client/Serveur Android)

Figure 4 : Adaptation dynamique de l'interface (Vue Étudiant à gauche, Vue Admin à droite).

3.2 Gestion du Réseau et Multithreading

Android interdit les opérations réseau sur le thread principal. J'ai donc utilisé des Thread séparés :

- **Côté Serveur :** Le serveur est "hybride". Il lance deux threads d'écoute permanents (un pour TCP, un pour UDP). Pour chaque connexion TCP acceptée, un nouveau thread dédié est instancié, garantissant la gestion parallèle de plusieurs clients.
- **Côté Client :** L'envoi se fait en arrière-plan. L'utilisateur a le choix du protocole via des boutons radio (TCP pour la fiabilité, UDP pour la rapidité).

L'utilisation de *ObjectOutputStream* et *ObjectInputStream* assure la reconstruction fidèle des objets Java à travers le réseau, quel que soit le protocole de transport choisi.

SAÉ 3.02 - Développement d'Applications Communicantes (Client/Serveur Android)

```java
private void runTCPServer() {
    try {
        ServerSocket serverSocket = new ServerSocket( port: 8080);
        updateLogs( msg: "Serveur TCP prêt (Port 8080)");
        while (isRunning) {
            Socket client = serverSocket.accept();
            new Thread(() -> {
                try {
                    ObjectInputStream ois = new ObjectInputStream(client.getInputStream());
                    MessagePacket packet = (MessagePacket) ois.readObject();

                    updateLogs( msg: "[TCP] " + packet.user + ": " + packet.content);
                    String reponse = processMessage(packet);

                    ObjectOutputStream oos = new ObjectOutputStream(client.getOutputStream());
                    oos.writeObject(reponse);
                    client.close();
                } catch (Exception e) { e.printStackTrace(); }
            }).start();
        }
    } catch (Exception e) { updateLogs( msg: "Err TCP: " + e.getMessage()); }
```

Figure 5 : Implémentation du Multithreading pour l'acceptation simultanée des clients.

4. Résultats et Tests

Les tests ont été effectués sous environnement émulé (Android Studio) et un téléphone android

Scénario 1 : Connexion Étudiant (Restreint)

- **Action :** Saisie de l'identifiant Etu1 et du mot de passe Roa42300.
- **Résultat :** Accès au menu principal. Le bouton "PANNEAU ADMIN (SERVEUR)" est **invisible**.
- **Test Réseau :** L'étudiant accède à l'interface Client, envoie un message en UDP. Le serveur reçoit bien : [UDP] Etu1 : Bonjour.

SAÉ 3.02 - Développement d'Applications Communicantes (Client/Serveur Android)

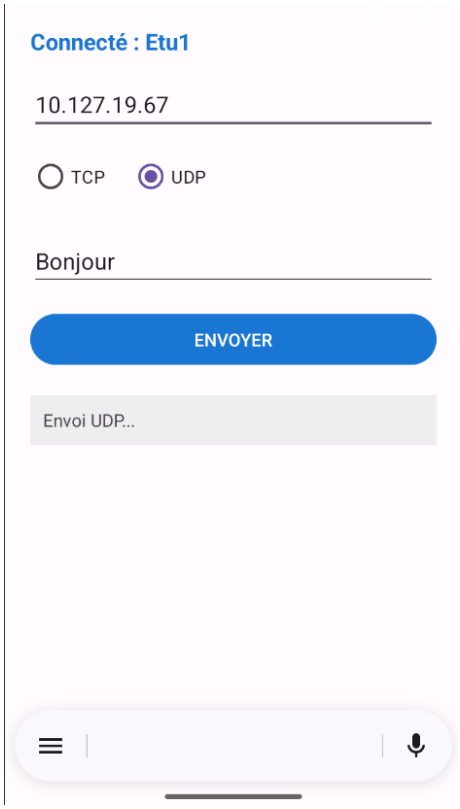

Figure 6 : Envoi d'un paquet UDP par l'utilisateur Etu1.

Scénario 2 : Connexion Administrateur (Privilégié)

- **Action :** Saisie de l'identifiant admin et du mot de passe root.
- **Résultat :** Accès au menu principal. Le bouton "PANNEAU ADMIN" est **visible**.
- **Test Serveur :** L'administrateur peut lancer le serveur. Il voit défiler les logs des connexions des autres utilisateurs (Etu1, Etu2) en temps réel.

Figure 7 : Réception et traitement des messages en temps réel sur le Serveur.

SAÉ 3.02 - Développement d'Applications Communicantes (Client/Serveur Android)

Scénario 3 : Sécurité (Mauvais mot de passe)

- **Action :** Tentative de connexion avec Etu1 et un mauvais mot de passe.
- **Résultat :** Un message "Toast" affiche "Identifiants incorrects" et l'accès au menu est bloqué.

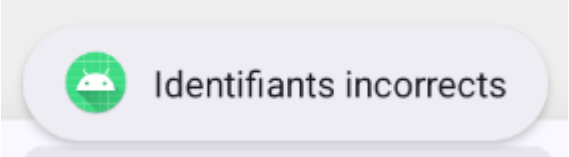

Figure 8 : Refus d'authentification en cas de mot de passe erroné.

5. Conclusion

Ce projet m'a permis de passer d'une simple simulation de socket à une architecture d'application cohérente. J'ai pu consolider mes compétences en programmation réseau Java (TCP/UDP, Sérialisation) tout en intégrant des concepts essentiels de développement mobile : gestion de session, passage de paramètres entre activités (Intent) et adaptation dynamique de l'interface utilisateur selon les droits. L'application finale est fonctionnelle, sécurisée et modulaire.